UNIT - IV

Manipulator Kinematics, Dynamics & Trajectory Planning

Automation and Robotics (23A03606) - handwritten study notes

Syllabus Topics

- Manipulator Kinematics: homogeneous transformations (rotation and translation), D-H notation, forward and inverse kinematics
- Manipulator Dynamics: differential transformations, Jacobians, Lagrange-Euler and Newton-Euler formulations
- Trajectory Planning: obstacle avoidance, path planning, skew motion, joint integrated motion, straight line motion

Why this unit matters: kinematics tells us WHERE the hand is for given joint angles (and vice versa), dynamics tells us WHAT forces/torques are needed, trajectory planning tells us HOW to move smoothly between points.

1. Manipulator Kinematics

1.1 What is kinematics?

Kinematics is the study of motion of the manipulator **without considering the forces** that cause the motion. It deals with position, velocity and acceleration of links and the end-effector.

- A manipulator is a chain of **rigid links** connected by **joints** (revolute R or prismatic P).
- Base of the arm is fixed, free end carries the **end-effector** (gripper / tool).
- Two basic problems are solved in kinematics:
  - **Forward kinematics**: joint variables ⟶ position and orientation of the end-effector

– **Inverse kinematics**: desired position and orientation of end-effector ⟶ joint variables

1.2 Forward vs Inverse kinematics

Point	Forward kinematics	Inverse kinematics
Given	Joint angles / displacements	Required end-effector pose
Find	End-effector position and orientation	Joint angles / displacements
Solution	Always unique, straightforward	May have many solutions or none
Method	Matrix multiplication of link transforms	Algebraic, geometric or numerical
Use	Display, simulation, checking	Control, motion programming

Key point: forward kinematics has a **unique** solution; inverse kinematics may have **multiple** solutions (elbow-up / elbow-down) or **no** solution if the point is outside the workspace.

2. Homogeneous Transformations

2.1 Why homogeneous matrices?

Rotation is a 3 x 3 matrix multiplication, but translation is an addition. To combine both into **one single matrix multiplication** we use a 4 x 4 **homogeneous transformation matrix**.

- A point (x, y, z) is written as a 4 x 1 column vector (x, y, z, 1)
- A 4 x 4 matrix T has the form: rotation 3 x 3 (top-left), position vector 3 x 1 (top-right), perspective 1 x 3 = 0 and scale factor = 1 (bottom row)

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & p_x \\ R_{21} & R_{22} & R_{23} & p_y \\ R_{31} & R_{32} & R_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Advantage**: a chain of transformations is simply the product $T = T_1 T_2$

T_3 ...

- **Caution**: matrix multiplication is **not commutative**, so the order matters.

2.2 Pure translation

Translating a point by (a, b, c) along x, y, z:

$$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Written as Trans(a, b, c). Inverse = Trans(-a, -b, -c).

2.3 Pure rotation about the axes

Using the shorthand c = cos(theta), s = sin(theta):

Rotation about X-axis, Rot(x, theta)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about Y-axis, Rot(y, theta)

$$\begin{bmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about Z-axis, Rot(z, theta)

$$\begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Properties of rotation matrices: columns are **orthogonal unit vectors**, determinant = +1, and the **inverse equals the transpose**: $R^{-1} = R^T$.

2.4 Inverse of a homogeneous matrix

For T with rotation R and position p:

$$T^{-1} = [\ R^T \ -R^T p\ ;\ 0\ 1\ ]$$

- Do not invert a 4 x 4 numerically, use this shortcut. It saves computation time in real-time controllers.

2.5 Composite transformations

- **Rotations about the current (moving) axes**: post-multiply (new transform on the RIGHT).
- **Rotations about the fixed (reference) axes**: pre-multiply (new transform on the LEFT).

Worked example 1

Point P = (7, 3, 2) is rotated 90 degrees about the Z-axis and then translated by (4, -3, 7). Find the new coordinates.

- Step 1: Rot(z, 90): c = 0, s = 1, so $x' = c\,x - s\,y = -3$, $y' = s\,x + c\,y = 7$, $z' = 2$.
- Step 2: Translation: $x'' = -3 + 4 = 1$, $y'' = 7 - 3 = 4$, $z'' = 2 + 7 = 9$.

Answer: P' = (1, 4, 9). (Order: T = Trans(4,-3,7) . Rot(z,90) applied to P.)

3. Degrees of Freedom and Joint Notation

- **Degrees of freedom (DOF)**: number of independent parameters needed to fix the pose of the end-effector. A free body in space has 6 DOF.
- A manipulator needs **6 joints** to reach any position AND orientation in its workspace (3 for arm and body, 3 for wrist).
- Joint types:

- **Revolute (R)**: rotation about an axis, variable = theta
- **Prismatic (P)**: sliding along an axis, variable = d
- Both have 1 DOF each.

4. Denavit-Hartenberg (D-H) Notation

4.1 Idea

Denavit and Hartenberg (1955) proposed a **systematic method** of attaching a coordinate frame to every link so that the relation between two successive links is described by just **four parameters**.

- Joint i connects link i-1 and link i.
- Frame {i} is attached to link i; Z-axis of frame {i-1} lies along the axis of joint i.

4.2 The four D-H parameters

Parameter	Name	Definition
$theta_i$	Joint angle	Rotation about z_{i-1} from x_{i-1} to x_i
d_i	Link offset	Distance along z_{i-1} from x_{i-1} to x_i
a_i	Link length	Distance along x_i from z_{i-1} to z_i
$alpha_i$	Link twist	Angle about x_i from z_{i-1} to z_i

For a **revolute** joint $theta_i$ is the **variable** (other three constant). For a **prismatic** joint d_i is the **variable**.

4.3 Procedure to assign frames (step by step)

- Step 3: Assign z_{i-1} along the axis of joint i.
- Step 5: Assign x_i along the common normal between z_{i-1} and z_i ($x_i = z_{i-1} \times z_i$).
- Step 7: Fill the **D-H table** with theta, d, a, alpha for each link.
- Step 8: Write the **A-matrix** for each link and multiply.

4.4 Link transformation matrix (A-matrix)

Going from frame {i-1} to frame {i} = Rot(z, theta) . Trans(0,0,d) .

Trans(a,0,0) . Rot(x, alpha):

$$\begin{bmatrix} c_i & -s_i c a_i & s_i s a_i & a_i c_i \\ s_i & c_i c a_i & -c_i s a_i & a_i s_i \\ 0 & s a_i & c a_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Here $c_i = \cos(\text{theta}_i)$, $s_i = \sin(\text{theta}_i)$, $ca_i = \cos(\text{alpha}_i)$, $sa_i = \sin(\text{alpha}_i)$.
- Forward kinematics of an n-joint arm:

$$T_0^n = A_1 A_2 A_3 \dots A_n$$

Memorise this matrix. It is asked in almost every exam. For a **6-DOF** arm the result $T = A_1A_2A_3A_4A_5A_6$ gives the hand pose (n, o, a, p).

5. Forward Kinematics

Link lengths L_1 and L_2; joint angles theta_1 and theta_2. Both joint axes are parallel and perpendicular to the plane.

Link	theta	d	a	alpha
1	theta_1	0	L_1	0
2	theta_2	0	L_2	0

Since alpha = 0 and d = 0 each A-matrix becomes:

Multiplying A_1 A_2 and using the identities $\cos(a+b) = \cos a \cos b - \sin a \sin b$, $\sin(a+b) = \sin a \cos b + \cos a \sin b$:

$$\begin{bmatrix} c_{12} & -s_{12} & 0 & L_1c_1+L_2c_{12} \\ s_{12} & c_{12} & 0 & L_1s_1+L_2s_{12} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- where $c_{12} = \cos(\theta_1 + \theta_2)$ and $s_{12} = \sin(\theta_1 + \theta_2)$.

End-effector position: $x = L_1\cos\theta_1 + L_2\cos(\theta_1+\theta_2)$ and $y = L_1\sin\theta_1 + L_2\sin(\theta_1+\theta_2)$. Orientation angle $\phi = \theta_1 + \theta_2$.

Numerical example

$L_1 = 2$ m, $L_2 = 1$ m, $\theta_1 = 30$ degrees, $\theta_2 = 60$ degrees.

- $x = 2\cos30 + 1\cos90 = 2(0.866) + 0 =$ **1.732 m**
- $y = 2\sin30 + 1\sin90 = 2(0.5) + 1 =$ **2.0 m**

6. Inverse Kinematics

6.1 Problem and difficulties

Given the desired hand pose (position + orientation) find $\theta_1 \dots \theta_n$. The equations are **non-linear** and **coupled**.

- Problems: **multiple solutions**, **no solution** (out of reach), **singularities**, and no general closed-form for all arms.

6.2 Geometric solution of 2-link planar arm

Target point (x, y). Use the cosine rule on the triangle formed by L_1, L_2 and the line from base to target.

$$x^2 + y^2 = L_1^2 + L_2^2 + 2\,L_1\,L_2\cos(\theta_2)$$

$$\cos(\theta_2) = (\,x^2 + y^2 - L_1^2 - L_2^2\,) / (\,2\,L_1\,L_2\,)$$

- Step 1: compute $\cos(\theta_2)$ as above. If its magnitude is greater than 1, the point is **unreachable**.
- Step 2: $\theta_2 = \arccos(.)$ gives **two solutions** (+ and -): **elbow-down** (+) and **elbow-up** (-).
- Step 3: find $\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2\sin(\theta_2), L_1 + L_2\cos(\theta_2))$.

> Always use **atan2(y, x)** rather than atan(y/x): it keeps the correct quadrant and avoids division by zero.

Worked example (inverse)

$L_1 = L_2 = 1$ m, target $(x, y) = (1, 1)$.

- $\cos(\theta_2) = (1 + 1 - 1 - 1) / (2) = 0$ so **θ_2 = +90 degrees** (or -90).
- $\theta_1 = \text{atan2}(1, 1) - \text{atan2}(1 \sin 90, 1 + 0) = 45 - 45 =$ **0 degrees**.
- Second solution: $\theta_2 = -90$, $\theta_1 = 45 - \text{atan2}(-1, 1) = 45 + 45 = 90$ degrees.

7. Manipulator Dynamics: Differential Transformations

- A **differential transformation** describes an infinitesimally small translation $d = (dx, dy, dz)$ and rotation $\delta = (\delta_x, \delta_y, \delta_z)$; for small angles $\sin(\delta) = \delta$, $\cos(\delta) = 1$.

> Small rotations about different axes **commute** (order does not matter), unlike finite rotations. Change of a frame: **dT = Delta . T**.

7.3 Differential motion of a manipulator

The change in hand position (dx, dy, dz) and rotation $(\delta_x, \delta_y, \delta_z)$ is related to the change in joint variables dq_i:

$$D = J \cdot dq$$

- $D = [dx\ dy\ dz\ \delta_x\ \delta_y\ \delta_z]^T$ is the **hand differential vector** (6 x 1).
- $dq = [dq_1 \dots dq_n]^T$ is the joint differential vector, J is the **Jacobian**.

8. The Jacobian

8.1 Definition

The **Jacobian** is the matrix that maps **joint velocities** to **end-effector**

(Cartesian) velocities:

$$V = J(q) \cdot q\text{-dot}$$

- $V = [vx\ vy\ vz\ omega_x\ omega_y\ omega_z]^T$ - linear and angular velocity of the hand.
- Size is 6 x n for an n-joint arm. It is **configuration dependent** (changes with q).
- Each column of J = effect of one joint's motion on the hand.

8.2 Jacobian of the 2-link planar arm

Start from the position equations and differentiate:

$$x = L_1c_1 + L_2c_{12}\quad y = L_1s_1 + L_2s_{12}$$

$$\begin{bmatrix} -L_1s_1 - L_2s_{12} & -L_2s_{12} \\ L_1c_1 + L_2c_{12} & L_2c_{12} \end{bmatrix}$$

- so that $[vx\ ;\ vy] = J \cdot [\theta_1\text{-dot}\ ;\ \theta_2\text{-dot}]$

Numerical example

$L_1 = L_2 = 1$ m, $\theta_1 = 0$, $\theta_2 = 90$ degrees. Joint speeds 1 rad/s each.

- $J = [\ -(0) - 1\ ,\ -1\ ;\ 1 + 0\ ,\ 0\ ] = [\ -1\ -1\ ;\ 1\ 0\ ]$
- $vx = (-1)(1) + (-1)(1) =$ **-2 m/s**, $vy = (1)(1) + 0 =$ **1 m/s**

8.3 Uses of the Jacobian

- **Velocity relation** between joint space and Cartesian space.
- **Force relation** by the principle of virtual work: **$\tau = J^T F$** (joint torques from the force at the hand).

8.4 Singularity from the Jacobian

For the planar arm: $\det(J) = L_1 L_2 \sin(\theta_2)$.

- $\det(J) = 0$ when **$\theta_2 = 0$ or 180 degrees** (arm fully stretched or folded back).
- At these points the hand cannot move along the arm direction; the Jacobian **loses rank**.

Key point: near a singularity the inverse Jacobian becomes huge, so joint velocities required are very large. Avoid singular configurations in path planning.

9. Dynamics of Manipulators

9.1 What is dynamics?

Dynamics studies the **relation between the forces/torques applied at the joints and the resulting motion** (positions, velocities, accelerations).

- **Forward dynamics**: given torques find the motion (used in simulation).
- **Inverse dynamics**: given the desired motion find the joint torques (used in control).

General equation of motion

$$\tau = M(q)\,\ddot{q} + C(q, \dot{q})\,\dot{q} + G(q)$$

- τ = joint torque vector, $M(q)$ = **inertia matrix** (symmetric, positive definite), C = **Coriolis and centrifugal** terms, G = **gravity** loading terms.

9.2 Lagrange-Euler (L-E) formulation

An **energy based** method. It uses Lagrangian $L = K - P$ where K = total **kinetic energy** and P = total **potential energy** of the arm.

$$\tau_i = \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i}$$

Steps of L-E method

- Step 1: Assign D-H coordinate frames to all links.
- Step 2: Find the position of the centre of mass of every link, using the A-matrices.
- Step 4: Calculate the **kinetic energy** K of each link: translational + rotational, then sum them.
- Step 7: Differentiate with respect to q_i and $\dot{q}_i$, then with time, to obtain τ_i.

9.3 L-E for a 2-link planar arm

Point masses m_1, m_2 at the ends of links of length L_1, L_2.

- Kinetic energy: $K = (1/2)\ m_1\ v_1^2 + (1/2)\ m_2\ v_2^2$
- Potential energy: $P = m_1\ g\ L_1\ s_1 + m_2\ g\ (\ L_1 s_1 + L_2 s_{12}\ )$

Resulting torque equations (standard form):

$$tau_1 = D_{11}\ theta_1\text{-ddot} + D_{12}\ theta_2\text{-ddot} + (Coriolis\ terms) + D_1$$

$$tau_2 = D_{21}\ theta_1\text{-ddot} + D_{22}\ theta_2\text{-ddot} + (Coriolis\ terms) + D_2$$

- $D_{11} = (m_1+m_2)\ L_1^2 + m_2\ L_2^2 + 2\ m_2\ L_1\ L_2\ c_2$
- $D_{12} = D_{21} = m_2\ L_2^2 + m_2\ L_1\ L_2\ c_2$, $D_{22} = m_2\ L_2^2$
- Gravity: $D_1 = (m_1+m_2)\ g\ L_1\ c_1 + m_2\ g\ L_2\ c_{12}$, $D_2 = m_2\ g\ L_2\ c_{12}$

Meaning of terms: D_{ii} = effective inertia at joint i, D_{ij} = coupling inertia, D_{iji} = **centripetal** force due to speed of joint j on joint i, D_{ijk} = **Coriolis** force from joints j and k, D_i = gravity load.

9.4 Newton-Euler (N-E) formulation

A **force balance** method: apply Newton's law F = m a and Euler's law N = I alpha + omega x I omega to each link.

- It is **recursive**: a forward recursion (base to tip) finds velocities and accelerations, then a backward recursion (tip to base) finds forces and torques.

Steps of N-E method

- Step 2: **Outward recursion** $i = 1$ to n: $omega_i$, $alpha_i$, linear acceleration of link i and its centre of mass.
- Step 3: Compute the inertial force $F_i = m_i\ a_{ci}$ and moment $N_i = I_i\ alpha_i + omega_i\ x\ (\ I_i\ omega_i\ )$.
- Step 4: **Inward recursion** $i = n$ to 1: $f_i = F_i + f_{i+1}$, and $n_i = N_i + n_{i+1}$ + position cross-force terms.
- Step 5: Joint torque $tau_i = n_i\ .\ z_{i-1}$ (revolute joint) or force $f_i\ .\ z_{i-1}$

(prismatic joint).

10. Trajectory Planning

10.1 Basic terms

- **Path**: the sequence of positions (and orientations) the hand passes through - a purely geometric description.
- **Trajectory**: the path **with time** - includes velocity and acceleration history.
- **Path planning**: deciding the geometric route, usually through via points between start and goal.
- **Trajectory planning**: computing the time function of each joint (or Cartesian coordinate) so the motion is smooth.
- Requirements: smooth motion, no jerks, joint velocity and acceleration within limits, avoid obstacles.

10.2 Joint space vs Cartesian space planning

Point	Joint space	Cartesian space
What is planned	Joint angles vs time	Hand position vs time
Computation	Light, no inverse kinematics online	Heavy, inverse kinematics at each step
Hand path	Unpredictable curve	Controlled (e.g. straight line)
Singularity	Not a problem	Can cause trouble
Use	Pick and place, fast moves	Welding, gluing, machining

10.3 Joint space - cubic polynomial

Conditions: start angle theta_0 at $t = 0$, end angle theta_f at $t = t_f$, start and end velocities = 0.

$$\text{theta}(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

- Boundary conditions: $\text{theta}(0) = \text{theta}_0$, $\text{theta}(t_f) = \text{theta}_f$, velocity(0)

= 0, velocity(t_f) = 0.

- Solution: a_0 = theta$_0$, a_1 = 0, a_2 = 3 (theta$_f$ - theta$_0$) / t_f^2, a_3 = -2 (theta$_f$ - theta$_0$) / t_f^3
- Velocity is parabolic, acceleration is **linear** (but not continuous at the end points).

Worked example (cubic)

A joint moves from 15 degrees to 75 degrees in 3 seconds, starting and ending at rest. Find the coefficients and the angle at t = 1 s.

- a_0 = 15, a_1 = 0
- a_2 = 3(75 - 15)/9 = 3(60)/9 = **20 deg/s^2**
- a_3 = -2(60)/27 = **-4.44 deg/s^3**
- theta(1) = 15 + 20(1) - 4.44(1) = **30.56 degrees**

The **quintic (5th order)** polynomial is used when acceleration at the ends must also be specified, giving smoother start and stop.

10.6 Cartesian-space trajectories

- **Straight line motion** (point-to-point in Cartesian space): the hand follows a straight line in space between the two end poses.
  - Divide the line into small equal steps. At every step compute the pose, then use **inverse kinematics** to get joint angles.
  - Position: $p(t) = p_0 + s(t)\,(p_f - p_0)$, where s goes from 0 to 1 smoothly.
  - Orientation is interpolated at the same time. Used in welding, sealing, cutting.
  - Cost: many inverse kinematics solutions and the path may pass through a singularity.
- **Skew motion**: the hand moves along a straight line **while its orientation also changes (rotates) at the same time**. It is also used in "slant" straight-line travel where the tool axis is skew to the path. The orientation change is interpolated about a fixed axis by an angle that

goes 0 ⟶ total angle.

- **Joint integrated (joint-interpolated) motion**: all joints start and stop **together**. The joint that has to travel the farthest sets the time, all other joints move proportionally slower. The hand path is a curve (not straight) but the motion is simple and quick.

Worked example (straight line)

Move the hand of a 2-link arm from (1, 1) to (1, 0.5) in 5 equal steps along a straight line.

- Step size in y = (0.5 - 1)/5 = -0.1 m, x constant.
- Points: (1, 1), (1, 0.9), (1, 0.8), (1, 0.7), (1, 0.6), (1, 0.5).
- For each point, solve inverse kinematics for $theta_1$, $theta_2$ and send to the controllers.

11. Obstacle Avoidance and Path Planning

11.2 Configuration space (C-space)

- A point in C-space = one complete set of joint values (q_1, ... q_n).
- Obstacles in workspace are mapped to **C-obstacles** (forbidden regions) in C-space.
- The planning task becomes: find a path for a **single point** from start to goal in the free C-space.

11.3 Common methods

- **Potential field**: goal attracts the arm, obstacles repel it; the arm slides down the combined potential.
  - Simple and fast, but can get trapped in **local minima**.
- **Via-point method (practical)**: user teaches extra via points so that the arm goes around the obstacle.

Key point: obstacle avoidance is easiest in **C-space**, and the final path is checked for collision of **every link**, not only the end-effector.

12. Quick Revision and Important Questions

Formula sheet

- Rot(z, theta): c -s / s c in the x-y block. Inverse of rotation = transpose.
- A-matrix = Rot(z,theta) Trans(0,0,d) Trans(a,0,0) Rot(x,alpha).
- Planar 2R: $x = L_1c_1 + L_2c_{12}$, $y = L_1s_1 + L_2s_{12}$.
- $\cos(\text{theta}_2) = (x^2 + y^2 - L_1^2 - L_2^2) / (2\ L_1\ L_2)$.
- Jacobian: $V = J\ \dot{q}$. Torque: $\tau = J^T F$. Singular if $\det(J) = 0$.
- Cubic: $a_2 = 3\ (\text{theta}_f - \text{theta}_0) / t_f^2$, $a_3 = -2\ (\text{theta}_f - \text{theta}_0) / t_f^3$.

Important questions

- Derive the forward kinematic equations of a 2-link planar manipulator.
- Derive the inverse kinematic solution for a 2-link arm. Why are there multiple solutions?
- What is the Jacobian? Derive it for a 2R arm and explain singularity.
- Explain Lagrange-Euler formulation. Derive the torque equations for a 2-link arm.
- Explain joint-space and Cartesian-space trajectory planning. Derive the cubic polynomial.

Exam tip: draw the diagram of the arm with link lengths and angles first, then show the D-H table, then the final answer. Neat diagrams earn marks.